

BHABBI / GETAWAY

Engineering Roadmap

Full-stack multiplayer card game — phase-by-phase build guide

Layer	Technology	Purpose
Backend API	FastAPI (Python)	Async HTTP + native WebSocket, auto OpenAPI docs
Database	PostgreSQL + SQLAlchemy	JSONB for card hands, async sessions via asyncpg
Migrations	Alembic	Version-controlled schema changes
Real-time	WebSocket (FastAPI)	Bidirectional events — no polling
Cache / Pub-Sub	Redis	Multi-instance WS coordination (Phase 5+)
Frontend	Next.js 14 App Router	React 18, client components for live game
Styling	Tailwind CSS	Rapid iteration on card & game UI
Identity	UUID token / localStorage	No login required — guest player system

Phase	Name	What gets built	Est.
1	Project Setup	FastAPI skeleton, DB, Alembic, config	1–2 days
2	Identity System	Guest token, players table, 3 endpoints	1 day
3	Room System	Create/join/leave/kick, 6 endpoints, 2 tables	2–3 days
4	Game Engine	Card logic, state machine, 4 HTTP endpoints, 4 tables	3–4 days
5	WebSocket Layer	Real-time events, connection manager, 15 WS events	3–4 days
6	Chat System	Persistent chat, system messages, 1 endpoint	1 day
7	Frontend Shell	Next.js setup, identity hook, lobby, waiting room	3–4 days
8	Game UI	Card hand, board, WS hook, chat panel, game over	5–7 days
9	Edge Cases	Disconnects, timers, host transfer, play again	2–3 days
10	Deployment	Docker Compose, Railway/Render, Redis pub-sub	2 days
Total			23–31 days

Game Rules — Bhabbi / Getaway

Players & Deck	2–6 players. Standard 52-card deck dealt equally. If not divisible, extra cards go to earlier seats.
First Turn	Whoever holds the Ace of Spades plays first and MUST play it as the opening card of the game.
Valid Play	Play a card with the same suit AND strictly higher rank than the top card. OR play any Ace (resets the round). OR play a 2 of any suit (beats everything — highest possible card).
Cannot Play	If a player cannot or does not play, they pick up the ENTIRE pile. The pile clears. The next player starts a completely fresh round — top_card becomes NULL and any card is valid.
Card Rank Order	$3 < 4 < 5 < 6 < 7 < 8 < 9 < 10 < J < Q < K < A < 2$. The 2 is the nuclear card — always wins.
Winning	First player to empty their hand wins and leaves the game. Second player to finish ranks 2nd, and so on.
The Bhabbi	The very last player still holding cards is the Bhabbi (loser). This is the central comedic element of the game.
Strategy Note	Picking up voluntarily (even when you have a valid card) is legal and can be strategic — accumulating cards for a 2 to dominate later.

Project Skeleton & Configuration

Repository
game-spec

Repository structure

```
bhabbi/  
├── backend/  
│   ├── main.py # FastAPI app entry point  
│   ├── database.py # Async engine + session factory  
│   ├── config.py # Pydantic Settings, reads .env  
│   ├── deps.py # FastAPI dependency functions (get_db, get_current_player)  
│   ├── models/  
│   │   ├── base.py # DeclarativeBase + UUID primary key mixin  
│   │   ├── schemas/ # Pydantic request/response models  
│   │   ├── routers/ # FastAPI routers – one per domain  
│   │   ├── services/ # Business logic (pure functions, no DB calls)  
│   │   ├── ws/ # WebSocket manager + router  
│   │   └── alembic/ # Migration files  
│   └── frontend/  
│       ├── app/ # Next.js App Router  
│       ├── components/  
│       ├── hooks/  
│       └── lib/  
├── docker-compose.yml  
└── .env
```

config.py — Pydantic Settings

```
from pydantic_settings import BaseSettings  
  
class Settings(BaseSettings):  
    DATABASE_URL: str  
    REDIS_URL: str = "redis://localhost:6379"  
    SECRET_KEY: str = "change-me-in-prod"  
    MAX_ROOM_SIZE: int = 6  
    TURN_TIMEOUT_SECONDS: int = 30  
  
class Config:  
    env_file = ".env"  
  
settings = Settings()
```

database.py — Async SQLAlchemy engine

```
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession  
from sqlalchemy.orm import sessionmaker, DeclarativeBase  
from .config import settings  
  
engine = create_async_engine(settings.DATABASE_URL, echo=False)  
AsyncSessionLocal = sessionmaker(engine, class_=AsyncSession,  
    expire_on_commit=False)  
  
class Base(DeclarativeBase):  
    pass
```

deps.py — Dependency injection

```
async def get_db() -> AsyncSession:
    async with AsyncSessionLocal() as session:
        yield session

async def get_current_player(
    x_player_token: str = Header(...),
    db: AsyncSession = Depends(get_db)
) -> Player:
    player = await db.scalar(
        select(Player).where(Player.token == x_player_token))
    if not player:
        raise HTTPException(401, "Invalid player token")
    player.last_seen = datetime.utcnow()
    await db.commit()
    return player
```

main.py — FastAPI application

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from .routers import identity, rooms, games
from .ws.router import ws_router

app = FastAPI(title="Bhabbi API")
app.add_middleware(CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_credentials=True, allow_methods=["*"], allow_headers=["*"])

app.include_router(identity.router, prefix="/api/identity")
app.include_router(rooms.router, prefix="/api/rooms")
app.include_router(games.router, prefix="/api/games")
app.include_router(ws_router)

@app.get("/api/health")
async def health(): return {"status": "ok"}
```

Alembic setup commands

```
pip install fastapi uvicorn sqlalchemy asyncpg alembic pydantic-settings redis

cd backend
alembic init alembic

# Edit alembic.ini: set sqlalchemy.url = your DATABASE_URL
# Edit alembic/env.py:
# from backend.database import Base
# target_metadata = Base.metadata

alembic revision --autogenerate -m "initial tables"
alembic upgrade head
```

Phase 1 — Done when:

- `uvicorn backend.main:app --reload` starts without errors

- `http://localhost:8000/docs` shows the OpenAPI Swagger UI
- Alembic migrations run cleanly against your Postgres instance
- `GET /api/health` returns `{"status": "ok"}`

Guest Identity System

No login re
backend m

How guest identity works

The flow is: user enters a nickname → POST /api/identity/register → backend generates a UUID token → frontend stores token in localStorage → every subsequent API call and WebSocket connection sends this token → backend validates it and resolves the Player record.

If the user clears localStorage or visits from a new device, they get a new identity. This is intentional — no account recovery is needed for a casual card game.

Table: players		
Column	Type	Description / Constraints
id	UUID PK	Internal identifier. Never sent to other players — only nicknames are shared.
token	UUID UNIQUE NOT NULL	The secret stored in the browser. Must be indexed for fast lookup on every request.
nickname	VARCHAR(32) NOT NULL	Display name chosen by user. 2–32 chars. Validated server-side.
created_at	TIMESTAMP	Set to NOW() by default. Never updated.
last_seen	TIMESTAMP	Updated on every authenticated request via the get_current_player dependency. Useful for analytics and stale cleanup.

One row per u

Endpoint 2.1 — POST /api/identity/register

POST	/api/identity/register	No authentication required
→	Request body: { "nickname": "Ali" } — validate 2–32 chars server-side, strip whitespace	
2	1. Generate UUID v4 as the player token using uuid.uuid4()	
3	2. Generate a second UUID v4 as the player id	
4	3. INSERT into players table. Wrap in try/except IntegrityError for the (astronomically rare) duplicate token collision — retry with a new UUID.	
5	4. Return { "player_token": "...", "player_id": "...", "nickname": "Ali" }	
→	Frontend action: localStorage.setItem("bhabbi_token", data.player_token)	
→	Error 422: nickname shorter than 2 chars or longer than 32 chars	

Endpoint 2.2 — GET /api/identity/me

GET	/api/identity/me	Header: X-Player-Token
-----	------------------	------------------------

1	1. Read X-Player-Token header (FastAPI Header(...) parameter)
2	2. SELECT * FROM players WHERE token = header_value. If not found → raise HTTPException(401)
3	3. UPDATE players SET last_seen = NOW() WHERE id = player.id
4	4. Return { "player_id": "...", "nickname": "Ali", "created_at": "..." }
→	Frontend calls this on every app load to validate the stored token
→	If 401 response: clear localStorage, set needsNickname = true, show NicknameModal

Endpoint 2.3 — PATCH /api/identity/me

PATCH	/api/identity/me	Header: X-Player-Token
→	Request body: { "nickname": "NewName" }	
2	1. Authenticate via get_current_player dependency	
3	2. Validate new nickname: 2–32 chars, strip whitespace	
4	3. UPDATE players SET nickname = ? WHERE id = player.id	
5	4. Return { "nickname": "NewName" }	
→	If player is currently in a room, the WS layer broadcasts a nickname change event to the room (implemented in Phase 5)	

Frontend — identity module (lib/identity.ts)

```
const TOKEN_KEY = "bhabbi_token"
const NICKNAME_KEY = "bhabbi_nickname"

export const getToken = () => localStorage.getItem(TOKEN_KEY)
export const saveToken = (t: string) => localStorage.setItem(TOKEN_KEY, t)
export const clearToken = () => localStorage.removeItem(TOKEN_KEY)
export const getNickname = () => localStorage.getItem(NICKNAME_KEY)
export const saveNickname = (n: string) => localStorage.setItem(NICKNAME_KEY, n)
```

Frontend — useIdentity hook (hooks/useIdentity.ts)

```
export function useIdentity() {
  const [player, setPlayer] = useState(null)
  const [loading, setLoading] = useState(true)
  const [needsNickname, setNeedsNickname] = useState(false)

  useEffect(() => {
    const token = getToken()
    if (!token) { setNeedsNickname(true); setLoading(false); return }
    api.get("/identity/me", token)
      .then(setPlayer)
      .catch(() => { clearToken(); setNeedsNickname(true) })
      .finally(() => setLoading(false))
  }, [])

  const register = async (nickname: string) => {
    const data = await api.post("/identity/register", { nickname })
    saveToken(data.player_token)
    saveNickname(nickname)
    setPlayer(data)
  }
}
```

```
setNeedsNickname(false)
}
return { player, loading, needsNickname, register }
}
```

Room System

Players cre
Rooms trac

Table: rooms			One row per r
Column	Type	Description / Constraints	
id	UUID PK	Internal ID used in all API paths and foreign keys.	
code	VARCHAR(6) UNIQUE	Human-readable join code e.g. "BHB4F2". Generated with random.choices(string.ascii_uppercase + digits, k=6). Add DB index.	
host_id	UUID FK → players	Who created the room. Only they can start the game and kick players. Changes if host leaves.	
status	ENUM	Values: 'waiting' 'playing' 'finished'. Only 'waiting' rooms accept new joins.	
max_players	SMALLINT default 4	2–6 players. Enforced on join.	
created_at	TIMESTAMP	For stale room cleanup cron (delete rooms older than 24h with no activity).	

Table: room_players			Join table bet
Column	Type	Description / Constraints	
id	UUID PK		
room_id	UUID FK → rooms	CASCADE DELETE so cleaning up a room removes all seat entries automatically.	
player_id	UUID FK → players	UNIQUE(room_id, player_id) — one entry per player per room. A player cannot be in two rooms simultaneously.	
seat	SMALLINT NOT NULL	0, 1, 2, 3, 4, 5. Seat 0 = host. Determines turn order. UNIQUE(room_id, seat).	
is_active	BOOLEAN default TRUE	Set FALSE when player leaves or is kicked. Seat stays reserved so turn order does not shift mid-game.	
joined_at	TIMESTAMP	Useful for ordering when assigning new host on host departure.	

Endpoint 3.1 — POST /api/rooms/create

POST	/api/rooms/create	Header: X-Player-Token
------	-------------------	------------------------

→	Body: { "max_players": 4 } — optional, default 4, valid range 2–6
2	1. Authenticate player via get_current_player dependency
3	2. Check if player already has an active room entry (status=waiting or playing). If yes → return 409 with existing room_id. Prevents duplicate rooms.
4	3. Generate 6-char uppercase alphanumeric code. SELECT to check uniqueness. Retry loop max 5 times on collision.
5	4. INSERT into rooms with status="waiting", host_id=player.id, max_players from body
6	5. INSERT into room_players with room_id, player_id, seat=0, is_active=TRUE
7	6. Return { room_id, code, status, max_players, host_id, players: [{player_id, nickname, seat}] }

Endpoint 3.2 — POST /api/rooms/join

POST	/api/rooms/join	Header: X-Player-Token
→	Body: { "code": "BHB4F2" } — uppercase the code server-side before lookup	
2	1. SELECT room WHERE code = upper(body.code). 404 if not found.	
3	2. If room.status != "waiting" → 400 "Game already in progress"	
4	3. COUNT active room_players WHERE room_id. If >= max_players → 422 "Room is full"	
5	4. SELECT room_players WHERE room_id AND player_id. If found and is_active=TRUE → 409 "Already in room". If is_active=FALSE → 403 "You were removed from this room".	
6	5. Find lowest unoccupied seat number. INSERT into room_players.	
7	6. Broadcast room_update WS event to all existing room members (wire this up in Phase 5)	
8	7. Return full room state including all players	

Endpoint 3.3 — GET /api/rooms/{room_id}

GET	/api/rooms/{room_id}	Header: X-Player-Token
1	1. Verify player is in the room via SELECT room_players. 403 if not a member.	
2	2. JOIN rooms + room_players + players to get full player list with nicknames	
3	3. Return { room_id, code, status, max_players, host_id, players: [{player_id, nickname, seat, is_active}] }	

Endpoint 3.4 — GET /api/rooms/public

GET	/api/rooms/public	No auth required
1	1. SELECT rooms WHERE status="waiting". JOIN to count active room_players. Filter where count < max_players.	
2	2. JOIN to get host nickname. Order by created_at DESC. Limit 20.	
3	3. Return { rooms: [{ room_id, code, player_count, max_players, host_nickname }] }	
→	Optional feature for v1. The primary join flow is by code — this is just a discovery list.	

Endpoint 3.5 — POST /api/rooms/{room_id}/leave

POST **/api/rooms/{room_id}/leave**

Header: X-Player-Token

- 1 1. Verify player is in room and is_active=TRUE
- 2 2. SET is_active=FALSE for this room_player row
- 3 3. If player was host AND status="waiting": find next active player by seat order, assign them as host. If no one remains → DELETE the room entirely.
- 4 4. If game is "playing": mark player inactive. Their turns will be auto-skipped (Phase 9 edge case handling).
- 5 5. Broadcast room_update WS event to remaining players

Endpoint 3.6 — POST /api/rooms/{room_id}/kick**POST** **/api/rooms/{room_id}/kick**

Header: X-Player-Token (host only)

- Body: { "player_id": "uuid-of-player-to-kick" }
- 2 1. Verify the requester is the room host. 403 if not.
 - 3 2. Verify room.status = "waiting". Cannot kick during a game.
 - 4 3. SET is_active=FALSE for the target player row
 - 5 4. Send kicked WS event { "type": "kicked" } to the kicked player specifically
 - 6 5. Broadcast room_update to all remaining players so their UI refreshes

Game Logic Engine

The heart of the engine.
Build and test logic.

Table: games			One row per room
Column	Type	Description / Constraints	
id	UUID PK		
room_id	UUID FK → rooms	One game per room at a time.	
started_at	TIMESTAMP		
ended_at	TIMESTAMP NULL	Set when game_state.phase = "finished".	
winner_id	UUID FK → players NULL	First player to empty their hand.	
bhabbi_id	UUID FK → players NULL	Last player still holding cards.	

Table: game_state			Updated atomically
Column	Type	Description / Constraints	
game_id	UUID PK FK → games	1:1 with games. The single source of truth for the live game.	
current_turn	UUID FK → players	Whose turn it is RIGHT NOW.	
pile	JSONB	Cards currently on the table: [{"suit":"spades","rank":"A"}, ...]. Empty array = fresh round.	
top_card	JSONB NULL	The card that must be beaten. NULL = fresh round, any card is valid.	
turn_started_at	TIMESTAMP	For 30-second timeout (Phase 9). Updated on every turn change.	
phase	ENUM	"in_progress" "finished"	

Table: player_hands			One row per player
Column	Type	Description / Constraints	
game_id	UUID FK → games	Part of composite PK.	
player_id	UUID FK → players	Part of composite PK. UNIQUE(game_id, player_id).	
cards	JSONB	Current hand as array: [{"suit":"hearts","rank":"K"}, ...]. Updated atomically with game_state.	
finished_at	TIMESTAMP NULL	Set when hand becomes empty. NULL = still playing.	
finish_rank	SMALLINT NULL	1=winner, 2=second, etc. Last player gets rank = total_players (they are the Bhabbi).	

Table: game_moves			Append-only
Column	Type	Description / Constraints	

id	UUID PK	
game_id	UUID FK → games	Indexed for replay and audit queries.
player_id	UUID FK → players	
move_type	ENUM	"play_card" "pickup" "timeout_pickup" "finish"
card_played	JSONB NULL	Only populated for move_type="play_card".
created_at	TIMESTAMP	Sequence by this for replay. Immutable — never updated.

Card representation and rank order

```
SUITS = ["spades", "hearts", "diamonds", "clubs"]
RANKS = ["3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K", "A", "2"]

# Rank order for comparison. 2 = 15 (highest). A = 14.
RANK_ORDER = {
    "3":3, "4":4, "5":5, "6":6, "7":7, "8":8, "9":9,
    "10":10, "J":11, "Q":12, "K":13, "A":14, "2":15
}

def create_deck() -> list[dict]:
    return [{"suit":s, "rank":r} for s in SUITS for r in RANKS]
# 4 suits x 13 ranks = 52 cards total
```

Game logic service — services/game_logic.py

```
import random

def shuffle_and_deal(player_ids: list) -> dict[str, list]:
    deck = create_deck()
    random.shuffle(deck)
    hands = {pid: [] for pid in player_ids}
    for i, card in enumerate(deck):
        hands[player_ids[i % len(player_ids)]].append(card)
    return hands # {player_id: [card, ...]}

def find_first_player(hands: dict) -> str:
    for pid, cards in hands.items():
        if {"suit": "spades", "rank": "A"} in cards:
            return pid
    raise ValueError("Ace of Spades not found in any hand")

def can_play(card: dict, top_card: dict | None) -> bool:
    if top_card is None:
        return True # Fresh round – anything goes
    if card["rank"] == "2":
        return True # 2 beats everything unconditionally
    if card["suit"] != top_card["suit"]:
        return False # Must match suit (unless it is an Ace)
    return RANK_ORDER[card["rank"]] > RANK_ORDER[top_card["rank"]]

def get_playable_cards(hand: list, top_card: dict | None) -> list:
    return [c for c in hand if can_play(c, top_card)]

def remove_card(hand: list, card: dict) -> list:
```

```

hand = list(hand)
hand.remove(card) # Raises ValueError if card not in hand
return hand

def next_active_player(
    current_id: str,
    seat_order: list[str], # player_ids ordered by seat
    finished: set[str],
    inactive: set[str]
) -> str:
    skip = finished | inactive
    n = len(seat_order)
    idx = seat_order.index(current_id)
    for i in range(1, n + 1):
        candidate = seat_order[(idx + i) % n]
        if candidate not in skip:
            return candidate
    raise ValueError("No active players remaining – game should have ended")

```

Endpoint 4.1 — POST /api/games/start

POST	/api/games/start	Header: X-Player-Token (host only)
→	Body: { "room_id": "uuid" }	
2	1. Verify caller == room.host_id. 403 if not.	
3	2. Count active room_players. Must be >= 2. 422 "Need at least 2 players" if not.	
4	3. Collect player_ids in seat order (ORDER BY seat ASC).	
5	4. Call shuffle_and_deal(player_ids) to get {player_id: [cards]}	
6	5. Call find_first_player(hands) to find who has Ace of Spades	
7	6. SET room.status = "playing"	
8	7. INSERT games row with room_id, started_at=NOW()	
9	8. INSERT game_state row: current_turn=first_player, pile=[], top_card=NULL, phase="in_progress"	
10	9. INSERT one player_hands row per player with their dealt cards	
11	10. Broadcast game_started WS event. Send hand_update privately to each player with their own hand.	
12	11. Return { game_id, your_hand: [...], current_turn: "uuid", top_card: null }	

Endpoint 4.2 — GET /api/games/{game_id}/state

GET	/api/games/{game_id}/state	Header: X-Player-Token
-----	----------------------------	------------------------

1	1. Verify player is in the game room
2	2. Fetch game_state + all player_hands
3	3. For other players: return only card COUNT, not card contents
4	4. For yourself: return full hand
→	Used for reconnect recovery in Phase 5. Not needed during normal play.
6	Return: { game_id, current_turn, top_card, pile_count, your_hand, players: [{player_id, nickname, card_count, finish_rank}] }

Endpoint 4.3 — POST /api/games/{game_id}/play

POST	/api/games/{game_id}/play	Header: X-Player-Token
→	Body: { "card": { "suit": "hearts", "rank": "K" } }	
2	1. Verify game_state.current_turn == player.id. 400 error code NOT_YOUR_TURN if not.	
3	2. Fetch player hand from player_hands table.	
4	3. Verify card exists in hand. 400 INVALID_CARD if not.	
5	4. Call can_play(card, game_state.top_card). 400 INVALID_PLAY if false.	
6	5. Call remove_card(hand, card). Save updated hand to player_hands.	
7	6. Append card to game_state.pile. Set game_state.top_card = card.	
8	7. If hand now empty: set player_hands.finished_at=NOW(), assign finish_rank. Log "finish" move.	
9	8. Call next_active_player() to find who goes next. Update game_state.current_turn. Update turn_started_at=NOW().	
10	9. If only 1 active player remains (the Bhabbi): set game_state.phase="finished", games.ended_at, games.bhabbi_id. Broadcast game_over.	
11	10. Log "play_card" move in game_moves. Broadcast card_played WS event to all in room.	

Endpoint 4.4 — POST /api/games/{game_id}/pickup

POST	/api/games/{game_id}/pickup	Header: X-Player-Token
1	1. Verify it is the player turn. 400 NOT_YOUR_TURN if not.	
2	2. (Optional) Check player has no valid card to play. Can skip — allow voluntary pickups.	
3	3. Append all cards in game_state.pile to player hand. Update player_hands.	
4	4. Set game_state.pile = [], game_state.top_card = NULL. Fresh round starts.	
5	5. Call next_active_player(). Update current_turn and turn_started_at.	
6	6. Log "pickup" move. Broadcast pickup WS event — include card COUNT picked up, NOT the cards themselves.	

Replace HT
shares one
channel.

WebSocket Real-time Layer

Connection manager — ws/manager.py

```
from fastapi import WebSocket
import json

class ConnectionManager:
    def __init__(self):
        # room_id -> { player_id -> WebSocket }
        self._rooms: dict[str, dict[str, WebSocket]] = {}

    async def connect(self, ws: WebSocket, room_id: str, player_id: str):
        await ws.accept()
        if room_id not in self._rooms:
            self._rooms[room_id] = {}
        self._rooms[room_id][player_id] = ws

    def disconnect(self, room_id: str, player_id: str):
        if room_id in self._rooms:
            self._rooms[room_id].pop(player_id, None)
        if not self._rooms[room_id]:
            del self._rooms[room_id]

    async def broadcast(self, room_id: str, msg: dict,
                        exclude: str | None = None):
        if room_id not in self._rooms: return
        dead = []
        for pid, ws in self._rooms[room_id].items():
            if pid == exclude: continue
            try:
                await ws.send_json(msg)
            except Exception:
                dead.append(pid)
        for pid in dead: self.disconnect(room_id, pid)

    async def send_to(self, room_id: str, player_id: str, msg: dict):
        ws = self._rooms.get(room_id, {}).get(player_id)
        if ws:
            try: await ws.send_json(msg)
            except: self.disconnect(room_id, player_id)

manager = ConnectionManager() # singleton – import everywhere
```

WebSocket endpoint — WS /ws/{room_id}

```
@ws_router.websocket("/ws/{room_id}")
async def game_ws(ws: WebSocket, room_id: str,
                  token: str = Query(...),
                  db: AsyncSession = Depends(get_db)):

    # 1. Authenticate
```

```
player = await get_player_by_token(db, token)
if not player:
    await ws.close(code=4001); return

# 2. Verify player is in room
rp = await get_room_player(db, room_id, player.id)
if not rp or not rp.is_active:
    await ws.close(code=4003); return

# 3. Connect
await manager.connect(ws, room_id, str(player.id))

# 4. Send full current state to this player (for reconnect)
state = await build_full_state(db, room_id, player.id)
await manager.send_to(room_id, str(player.id),
{"type": "connected", **state})

# 5. Notify others that this player is present
await manager.broadcast(room_id,
{"type": "player_connected", "player_id": str(player.id),
"nickname": player.nickname}, exclude=str(player.id))

try:
    while True:
        data = await ws.receive_json()
        await handle_message(db, room_id, player, data)
    except WebSocketDisconnect:
        manager.disconnect(room_id, str(player.id))
        await manager.broadcast(room_id,
{"type": "player_disconnected",
"player_id": str(player.id),
"nickname": player.nickname})
```

Client → Server messages

start_game

client → server

Sent by host only. Server validates caller is host, calls game start logic from Phase 4, broadcasts game_started to room.

```
{ "type": "start_game" }
```

play_card

client → server

Server runs Phase 4 play logic. On success → broadcast card_played. On error → send error event to this player only.

```
{ "type": "play_card", "card": {"suit": "spades", "rank": "A"} }
```

pickup

client → server

Run pickup logic. Send hand_update privately to this player. Broadcast pickup to all with card COUNT only, never card contents.

```
{ "type": "pickup" }
```

chat

client → server

Validate max 500 chars, strip whitespace. Save to chat_messages. Broadcast to all in room including sender.

```
{ "type": "chat", "text": "bhai bhabbi kar diya!" }
```

ping

client → server

Reply immediately with pong. Frontend sends this every 30s to keep connection alive through proxies and load balancers.

```
{ "type": "ping" }
```

play_again

client → server

Host only. After game_over. Server resets game state, deals new hands, broadcasts game_started.

```
{ "type": "play_again" }
```

Server → Client messages**connected**

server → you only

Sent immediately on connect/reconnect. Full state so frontend can rebuild UI from scratch.

```
{ "type": "connected",
  "room": { ...room fields... },
  "game": null | {
    "current_turn": "uuid", "top_card": null | {...},
    "pile_count": 0, "your_hand": [...],
    "players": [{player_id, nickname, card_count, finish_rank}]
  }
}
```

game_started

server → broadcast to room

```
{ "type": "game_started", "game_id": "uuid", "current_turn": "uuid",
  "player_order": [{player_id, nickname, seat, card_count}] }
```

// Also sends privately to each player:

```
{ "type": "hand_update", "your_hand": [{suit, rank}, ...] }
```

card_played

server → broadcast to room

Never broadcast other players card contents. Only card counts and the card just played.

```
{ "type": "card_played", "player_id": "uuid", "nickname": "Ali",
  "card": {"suit": "spades", "rank": "A"},
  "new_top_card": {"suit": "spades", "rank": "A"},
  "pile_count": 3, "next_turn": "uuid",
  "card_counts": {"uuid1": 7, "uuid2": 12} }
```

hand_update

server → you only

Sent after every change to your hand — after deal, after playing, after pickup.

```
{ "type": "hand_update", "your_hand": [{"suit": "hearts", "rank": "K"}, ...] }
```

pickup

server → broadcast to room

```
{ "type": "pickup", "player_id": "uuid", "nickname": "Ali",  
  "cards_picked": 6, "next_turn": "uuid",  
  "card_counts": {"uuid1": 18, "uuid2": 12} }
```

player_finished

server → broadcast to room

```
{ "type": "player_finished", "player_id": "uuid",  
  "nickname": "Ali", "finish_rank": 1 }
```

game_over

server → broadcast to room

```
{ "type": "game_over", "winner_id": "uuid", "bhabbi_id": "uuid",  
  "results": [{player_id, nickname, finish_rank}] }
```

error

server → you only

```
{ "type": "error",  
  "code": "NOT_YOUR_TURN" | "INVALID_CARD" | "INVALID_PLAY"  
  | "GAME_NOT_STARTED" | "NOT_HOST",  
  "message": "Human readable error message" }
```

room_update

server → broadcast to room

```
{ "type": "room_update",  
  "players": [{player_id, nickname, seat, is_active}],  
  "host_id": "uuid", "status": "waiting" }
```

turn_timer

server → broadcast to room

Broadcast every 5s during a turn so all frontends can show a countdown.

```
{ "type": "turn_timer", "player_id": "uuid", "seconds_left": 25 }
```

kicked

server → you only

Frontend should show a toast and redirect to lobby on receiving this.

```
{ "type": "kicked", "reason": "Host removed you from the room" }
```

Chat System

Persistent i
automatica

Table: chat_messages		
Column	Type	Description / Constraints
id	UUID PK	
room_id	UUID FK → rooms	Indexed. Chat is room-scoped — persists during waiting and playing phases.
player_id	UUID FK → players NULL	NULL for system messages (game narration).
text	VARCHAR(500) NOT NULL	Stripped and trimmed server-side before saving.
is_system	BOOLEAN default FALSE	TRUE for auto-generated game narration messages. Styled differently in frontend.
sent_at	TIMESTAMP default NOW()	Indexed for paginated history queries.

Stores all cha

Endpoint 6.1 — GET /api/rooms/{room_id}/chat

GET	/api/rooms/{room_id}/chat	Header: X-Player-Token
→	Query params: ?limit=50&before;=ISO_TIMESTAMP (for pagination)	
2	1. Verify player is in room	
3	2. SELECT chat_messages WHERE room_id = ? AND sent_at < before ORDER BY sent_at DESC LIMIT limit	
4	3. JOIN players to get nickname for each non-system message	
5	4. Return in chronological order (reverse the DESC query result before returning)	
6	5. Return { messages: [{id, player_id, nickname, text, is_system, sent_at}], has_more: bool }	
→	Frontend calls this on room join to pre-load recent history before WS connects	

System messages — auto-generated by server

```
async def system_msg(db, room_id: str, text: str):
    """Insert system message to DB and broadcast to room."""
    msg = ChatMessage(room_id=room_id, player_id=None,
text=text, is_system=True)
    db.add(msg)
    await db.commit()
    await manager.broadcast(room_id, {
        "type": "chat", "player_id": None, "nickname": None,
        "text": text, "is_system": True, "sent_at": msg.sent_at.isoformat()
    })

# Call these after game events:
# game_started -> "Game started! Ali has the Ace of Spades."
# card_played -> "Ali played Ace of Hearts – fresh round!"
```

```
# card "2" played-> "Bilal played 2 of Clubs – nuclear card!"  
# pickup -> "Sara picked up 6 cards"  
# player_finished-> "Umar finished in position #1!"  
# game_over -> "Hamza is the Bhabbi! Game over."
```

Frontend Shell — Lobby & Waiting Room

Next.js app
room. No c

Next.js project structure

```
frontend/
  ■■■ app/
  ■ ■■■ layout.tsx # Root layout – wraps IdentityProvider
  ■ ■■■ page.tsx # Lobby (create / join room)
  ■ ■■■ room/[code]/
  ■ ■■■ page.tsx # WaitingRoom + GameBoard (client component)
  ■■■ components/
  ■ ■■■ NicknameModal.tsx # Blocks UI until nickname is set
  ■ ■■■ Lobby.tsx # Create room, join by code, public list
  ■ ■■■ WaitingRoom.tsx # Seat grid, player list, start button
  ■ ■■■ game/
  ■ ■■■ GameBoard.tsx # Center pile + turn indicator
  ■ ■■■ PlayerHand.tsx # Your cards (Phase 8)
  ■ ■■■ OpponentSeat.tsx
  ■ ■■■ GameOverModal.tsx
  ■■■ components/chat/
  ■ ■■■ ChatPanel.tsx
  ■■■ hooks/
  ■ ■■■ useIdentity.ts
  ■ ■■■ useGameSocket.ts # Phase 8
  ■■■ lib/
  ■■■ api.ts # Typed fetch wrapper
  ■■■ identity.ts # localStorage helpers
```

lib/api.ts — typed fetch wrapper

```
const BASE = process.env.NEXT_PUBLIC_API_URL

export const api = {
  get: async (path: string, token?: string) => {
    const r = await fetch(BASE + path, {
      headers: token ? { "X-Player-Token": token } : {}
    })
    if (!r.ok) throw await r.json()
    return r.json()
  },
  post: async (path: string, body: any, token?: string) => {
    const r = await fetch(BASE + path, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        ...(token ? { "X-Player-Token": token } : {})
      },
      body: JSON.stringify(body)
    })
    if (!r.ok) throw await r.json()
    return r.json()
  },
}
```

WaitingRoom.tsx — what to build

The waiting room is the first interactive screen players share. Build these features:

WaitingRoom component checklist

- Display the room code prominently in a large monospace font so players can share it easily
- Show a seat grid: each seat as a card — occupied seats show the player nickname, empty seats show "Waiting..."
- Host sees a Start Game button, enabled only when ≥ 2 players are active. Show a disabled state with tooltip "Need at least 2 players"
- Host sees a kick button (X icon) on each other player seat card
- Connect the WebSocket here and listen for room_update events to refresh the player list without polling
- When WS sends kicked event to you: show a toast notification, then redirect to lobby after 2 seconds
- Show a copy-to-clipboard button next to the room code

Game UI — Cards, Board & Live Play

The actual
cards or pic

useGameSocket hook — hooks/useGameSocket.ts

```
export function useGameSocket(roomId: string) {
  const token = getToken()
  const wsRef = useRef<WebSocket>()

  const [gameState, setGameState] = useState<GameState|null>(null)
  const [hand, setHand] = useState<Card[]>([])
  const [messages, setMessages] = useState<ChatMsg[]>([])
  const [connected, setConnected] = useState(false)

  useEffect(() => {
    const WS_URL = process.env.NEXT_PUBLIC_WS_URL
    const ws = new WebSocket(`${WS_URL}/ws/${roomId}?token=${token}`)
    wsRef.current = ws

    ws.onopen = () => setConnected(true)
    ws.onclose = (e) => {
      setConnected(false)
      if (e.code !== 1000) setTimeout(reconnect, 2000) // auto-reconnect
    }
    ws.onmessage = (e) => {
      const msg = JSON.parse(e.data)
      switch(msg.type) {
        case "connected": handleConnected(msg); break
        case "game_started": setGameState(msg); break
        case "card_played": handleCardPlayed(msg); break
        case "hand_update": setHand(msg.your_hand); break
        case "pickup": handlePickup(msg); break
        case "player_finished": handleFinish(msg); break
        case "game_over": handleGameOver(msg); break
        case "chat": setMessages(m => [...m, msg]); break
        case "room_update": handleRoomUpdate(msg); break
        case "error": toast.error(msg.message); break
        case "turn_timer": setSecondsLeft(msg.seconds_left); break
      }
    }

    // Ping every 30s to keep connection alive
    const pingInterval = setInterval(() => {
      ws.readyState === 1 && ws.send(JSON.stringify({type:"ping"})), 30000
    }, 30000)
    return () => { clearInterval(pingInterval); ws.close(1000) }
  }, [roomId])

  const playCard = (card: Card) => {
    wsRef.current?.send(JSON.stringify({type:"play_card", card}))
  }
  const pickup = () => {
    wsRef.current?.send(JSON.stringify({type:"pickup"}))
  }
  const sendChat = (text: string) => {
    wsRef.current?.send(JSON.stringify({type:"chat", text}))
  }
  const startGame = () => {
    wsRef.current?.send(JSON.stringify({type:"start_game"}))
  }
}
```

```
return { gameState, hand, messages, connected,
playCard, pickup, sendChat, startGame }
}
```

PlayerHand.tsx — card display logic

PlayerHand component implementation

- Render cards in a horizontal fan at the bottom. Apply slight CSS rotation to each card based on its index position for arc effect.
- Implement `can_play()` in TypeScript too (same logic as backend) to highlight playable cards client-side without a round-trip.
- Playable cards: bright border, cursor pointer, slight scale-up on hover.
- Non-playable cards: opacity 0.4, cursor not-allowed. Do not let them be clicked.
- On card click: only fires if it is your turn AND card is playable. Send `play_card` WS message. Optimistically remove from hand immediately. Add it back if server sends an error response.
- Card component: suit symbols rendered as Unicode (spades, hearts, diamonds, clubs). Red fill for hearts and diamonds. Black for spades and clubs.
- Show rank in top-left and bottom-right of each card face.

GameBoard.tsx — center table

GameBoard component implementation

- Pile display: show top card face-up in the center of the screen. If pile > 1, fan 2–3 card backs behind it for visual depth.
- Fresh round indicator: when `top_card` is null, show a subtle "Play anything" text in the pile area.
- Turn indicator: pulsing ring or colored border around the active player seat. Show "Your turn!" banner if it is you.
- Pick Up button: large red button, enabled only when it is your turn. Clicking calls `pickup()`. Optimistically clear pile in UI and show cards flying to hand animation.
- Card counts: small badge on each opponent seat showing remaining card count. Updates on every WS event.
- Turn timer: show countdown ring around the active player avatar when timer is under 10 seconds.

ChatPanel.tsx — sidebar

ChatPanel component implementation

- Fixed right sidebar, 300px wide on desktop. Full-screen overlay triggered by a button on mobile.
- Pre-load chat history on mount: `GET /api/rooms/{room_id}/chat?limit=50`.
- System messages: gray italic text, no avatar, slightly smaller font.
- Player messages: player initial avatar circle + nickname bold + message text.

- Input at bottom. Send on Enter key (not Shift+Enter which inserts newline). Character counter showing X/500.
- Auto-scroll to bottom when new message arrives ONLY if user is within 100px of the bottom — do not hijack scroll if they are reading older messages.

Edge Cases, Timers & Reconnection

The game s
misbehave.

Player disconnects mid-game

1. On WebSocketDisconnect: broadcast player_disconnected to room. Do NOT immediately end the game.
2. Start a 60-second reconnect window. Store disconnect_at timestamp in Redis or an in-memory dict keyed by player_id.
3. If they reconnect within 60s: send full connected state event. They resume normally. Broadcast player_reconnected.
4. If 60s elapses and it is their turn: trigger auto-pickup (same logic as manual pickup). Broadcast with reason: "timeout".
5. If 60s elapses and it is not their turn: continue game normally, skip them each time their turn comes.

Turn timer — 30 seconds per turn

1. On every turn change: set game_state.turn_started_at = NOW(). Create an asyncio.Task that fires after 30s.
2. Every 5 seconds broadcast turn_timer event so all frontends show a countdown.
3. If timer fires: run auto-pickup for the current player. Log as timeout_pickup move type.
4. Cancel the old task and restart on every move. Store tasks in a dict keyed by game_id.
5. Implementation: asyncio.create_task(turn_timer_coro(game_id, player_id, 30))

Host leaves room — pre-game

1. Find next active player by seat order: seat 1 first, then 2, etc.
2. UPDATE rooms SET host_id = next_player.player_id
3. Broadcast room_update with new host_id so frontend updates the Start Game button visibility.
4. If no other players remain: DELETE the room entirely.

Host leaves mid-game

1. Host has no special powers during a game — no need to transfer host role.
2. Mark host player inactive. Skip their turns. Game continues normally.
3. If only 1 active player remains after host leaves: trigger game over with that player as Bhabbi.

Play Again flow

1. Host sends { "type": "play_again" } WS event after game_over.
2. Verify game phase = "finished". 400 if not.
3. INSERT new game row. Clear game_state (reset pile, top_card, current_turn). Deal new hands.
4. SET room.status = "playing". Broadcast game_started with new game data.
5. Send private hand_update to each player with their new hand.
6. Frontend GameOverModal listens for game_started event: close modal, reset game UI.

All players disconnect simultaneously

1. Room and game state remain in DB.
2. If all reconnect within 60s: game resumes normally.
3. If no one reconnects: stale cleanup cron (hourly) marks game as abandoned.
4. Room status set to "finished" by cleanup job.

Only one active player remains (forced Bhabbi)

1. After every pickup, disconnect, or auto-skip: check if only 1 non-finished, active player remains.
2. If yes: they are the Bhabbi by default. SET games.bhabbi_id. SET game_state.phase="finished".
3. Broadcast game_over immediately. No further turns.

Stale room cleanup cron

1. Add a background task (asyncio loop or APScheduler) running every hour.
2. DELETE rooms WHERE status="waiting" AND created_at < NOW() - INTERVAL 24 hours AND no active WS connections.
3. DELETE rooms WHERE status="playing" AND game_state.updated_at < NOW() - INTERVAL 3 hours (abandoned games).
4. CASCADE deletes handle room_players, game_state, player_hands automatically.

Docker, Deployment & Production

Containeriz
Deploy bac

docker-compose.yml — full development stack

```
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: bhabbi
      POSTGRES_USER: bhabbi
      POSTGRES_PASSWORD: bhabbi
    ports: ["5432:5432"]
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U bhabbi"]
      interval: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

  backend:
    build: ./backend
    environment:
      DATABASE_URL: postgresql+asyncpg://bhabbi:bhabbi@db/bhabbi
      REDIS_URL: redis://redis:6379
    ports: ["8000:8000"]
    depends_on:
      db: {condition: service_healthy}
      redis: {condition: service_started}
    command: uvicorn backend.main:app --host 0.0.0.0 --port 8000 --reload
    volumes: [./backend:/app]

  frontend:
    build: ./frontend
    environment:
      NEXT_PUBLIC_API_URL: http://localhost:8000/api
      NEXT_PUBLIC_WS_URL: ws://localhost:8000
    ports: ["3000:3000"]
    depends_on: [backend]

volumes:
  pgdata:
```

backend/Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "backend.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Redis pub-sub for multi-instance WebSocket

When running 2+ backend instances (for reliability/scale), a WebSocket client connected to instance A cannot be reached by code running on instance B. Redis pub-sub bridges them.

```
import redis.asyncio as aioredis

redis = aioredis.from_url(settings.REDIS_URL)

async def broadcast_via_redis(room_id: str, msg: dict):
    """Publish to Redis – all instances will receive and deliver."""
    await redis.publish(f"room:{room_id}", json.dumps(msg))

async def start_redis_subscriber(room_id: str):
    """Each instance subscribes to rooms it has live clients for."""
    pubsub = redis.pubsub()
    await pubsub.subscribe(f"room:{room_id}")
    async for message in pubsub.listen():
        if message["type"] == "message":
            data = json.loads(message["data"])
            # Deliver to local WS clients only
            await manager.broadcast(room_id, data)
```

Production environment variables

```
# Backend .env
DATABASE_URL=postgresql+asyncpg://user:pass@host/bhabbi
REDIS_URL=redis://redis-host:6379
SECRET_KEY=a-very-long-random-string-generated-with-openssl-rand
MAX_ROOM_SIZE=6
TURN_TIMEOUT_SECONDS=30

# Frontend .env.production
NEXT_PUBLIC_API_URL=https://api.yourdomain.com/api
NEXT_PUBLIC_WS_URL=wss://api.yourdomain.com
```

Production go-live checklist

Before going live

- Change CORS allow_origins from * to your exact frontend domain only
- Set SECRET_KEY to a long random string. Generate with: openssl rand -hex 32
- Use HTTPS and WSS in production. WebSocket over plain HTTP is blocked by browsers on HTTPS pages.
- Set DB_POOL_SIZE and DB_MAX_OVERFLOW on SQLAlchemy engine for production load.
- Add rate limiting on POST /api/rooms/create (max 5 per minute per IP) to prevent spam.
- Run alembic upgrade head as part of your deployment process, not inside the Dockerfile.
- Set up periodic PostgreSQL backups. Railway and Render both have this built in.
- Add sentry-sdk error tracking to both backend (FastAPI middleware) and frontend (Next.js).
- Configure health check endpoint for your load balancer / deploy platform.

- Test WebSocket reconnection manually by disabling network for 30 seconds mid-game.

Complete Endpoint Reference

Method	Path	Auth	Phase	Description
POST	/api/identity/register	None	2	Create guest identity, get player token
GET	/api/identity/me	Token	2	Validate token, get my profile
PATCH	/api/identity/me	Token	2	Update nickname
POST	/api/rooms/create	Token	3	Create a room, get 6-char code
POST	/api/rooms/join	Token	3	Join room by code
GET	/api/rooms/{room_id}	Token	3	Get room details and player list
GET	/api/rooms/public	None	3	List open public rooms
POST	/api/rooms/{room_id}/leave	Token	3	Leave a room
POST	/api/rooms/{room_id}/kick	Token (host)	3	Kick a player (host only, pre-game)
POST	/api/games/start	Token (host)	4	Deal cards, start the game
GET	/api/games/{game_id}/state	Token	4	Get current game state (reconnect)
POST	/api/games/{game_id}/play	Token	4	Play a card (HTTP fallback)
POST	/api/games/{game_id}/pickup	Token	4	Pick up pile (HTTP fallback)
GET	/api/rooms/{room_id}/chat	Token	6	Load chat history (paginated)
WS	/ws/{room_id}?token=...	Query param	5	Primary real-time connection
GET	/api/health	None	1	Health check for load balancer